

Agentic Compliance Auditor

A Build-Up Guide: From First Principles to Deep Architecture

An AI assistant that audits documents against GDPR, HIPAA, and SOC2 using autonomous agents that plan, retrieve, reason, and self-check before returning a cited answer.

Author	Balaji V
Stack	LangGraph · OpenAI gpt-4o-mini · ChromaDB · FastAPI · Streamlit · Redis · Neo4j
Purpose	Understand the system end-to-end — what was built and why

00 What's Inside

01 The Problem & The Idea

Why generic LLMs fail at compliance, and the core insight

02 Concepts You Need First

RAG, embeddings, vector search, and what 'agentic' means

03 The Big Picture

How a question flows through the whole system

04 Step 1 — Ingestion

Turning regulatory PDFs into searchable chunks

05 Step 2 — Retrieval

Hybrid semantic + keyword search with citations

06 Step 3 — The Agent

LangGraph state machine: retrieve, generate, reflect

07 Step 4 — Memory & Graph

Redis sessions and the Neo4j knowledge graph

08 Step 5 — Serving It

FastAPI endpoints and the Streamlit UI

09 Evaluation

Measuring quality with RAGAS

10 Design Decisions & Lessons

The 'why' behind the choices, and what was hard

11 Glossary

Every key term in one place

01 The Problem & The Idea

Compliance teams spend hours cross-referencing regulatory documents to answer questions like “Do our data-retention policies meet GDPR Article 17?” or “What exactly does HIPAA require for PHI access logs?”. The work is slow, but the deeper problem is trust: a generic large language model will happily invent a plausible-sounding regulatory detail and cite nothing. In compliance, every claim must be traceable to a source — an unsourced answer is worse than no answer.

The core insight

Don't ask the model to *know* the regulations. Ask it to *reason over* the specific regulatory text you hand it, force it to cite, and then make it check its own work before answering. That single shift — from a model that recalls to a system that retrieves, grounds, and self-verifies — is the whole project.

What the system does, in one breath

You ask a compliance question. The system (1) searches your regulatory documents using both semantic and keyword retrieval, (2) drafts an answer grounded only in what it found, (3) critiques its own draft — if citations are missing or the answer is vague, it tries again — and (4) returns the final answer with specific article and section references. Everything except the LLM call runs on your machine.

A key privacy property

Documents, embeddings, and session state never leave your machine. Only the composed prompt (your question plus the retrieved chunks) is sent to OpenAI for the generation step. The vector database, the embedding model, and conversation history are all local.

02 Concepts You Need First

Before tracing the code, here are the four ideas the whole system rests on. If these click, everything else is just plumbing.

1. Embeddings — turning meaning into numbers

An embedding model reads a piece of text and outputs a fixed-length list of numbers (a vector) that captures its meaning. Texts with similar meaning land close together in this number-space, even if they share no words. This project uses **all-MiniLM-L6-v2**, which produces 384-dimensional vectors and runs locally as a library call — there is no embedding server to deploy.

2. Vector search — finding by meaning

Once every chunk of your documents is an embedding, answering “what’s relevant to this question?” becomes “which stored vectors are closest to the question’s vector?”. **ChromaDB** stores the vectors and answers that nearest-neighbour query using cosine similarity. This is *semantic* search — it finds conceptually related text even when the wording differs.

3. RAG — Retrieval-Augmented Generation

RAG is the pattern of retrieving relevant text first, then asking the LLM to answer *using that text*. The retrieved chunks become the model’s evidence, which is what makes citations possible and hallucinations rare. A plain RAG pipeline is two steps: retrieve, then generate.

4. “Agentic” — the step beyond plain RAG

An *agent* doesn’t just run a fixed pipeline; it can decide what to do next. Here, after generating a draft the system runs a **reflection** step that judges whether the answer is well-grounded and properly cited. If not, it loops back and retrieves again. That decision-and-loop — “is this good enough, or should I try harder?” — is what earns the word *agentic*. A plain pipeline can’t do it; it stops after the first draft.

The one-sentence mental model

Plain RAG = retrieve → generate. This project = retrieve → generate → reflect → (maybe loop back). The reflection loop is the difference between a search box and an agent.

03 The Big Picture

Here is the full journey of a single question, from the HTTP request to the cited answer. Read it top to bottom — each later section zooms into one of these stages.

```
User question
  | HTTP POST /query { query, session_id }
  v
FastAPI endpoint
  v
ComplianceAgent --- LangGraph StateGraph -----+

Step 1. RETRIEVE
  HybridRetriever
    - Semantic search -> ChromaDB (384-dim vectors)
    - Keyword boost   -> term matching over chunk text
    => top-k chunks + similarity scores + citations

Step 2. GENERATE
  system prompt + question + retrieved chunks
    -> OpenAI gpt-4o-mini => draft answer

Step 3. REFLECT (conditional)
  draft answer -> reflection LLM call
    grounded + cited? yes -> finalize
                      no  -> loop back to RETRIEVE -----+
                              (max N iterations)

  v
Final answer + citations
  |
  +--> Redis (session history; in-memory fallback)
  +--> Neo4j (regulation -> article -> requirement; optional)
  v
HTTP response { answer, sources[], citations[], iterations }
```

Three things the diagram makes explicit

- **The agent is sequential, not parallel.** Retrieve finishes before generate starts; generate finishes before reflect starts. The arrows are data flow, not concurrent threads.
- **Embeddings are a library call, not a service.** sentence-transformers runs in-process inside the retriever — nothing extra to host.
- **Reflection is the only optional, looping part.** Everything else runs exactly once per iteration; reflection is what can send the state back to the start.

04 Step 1 — Ingestion: Documents into Chunks

Before any question can be answered, the regulatory documents must be loaded, split into chunks, embedded, and stored. This is a one-time offline step you run with `python setup.py`. Code lives in `src/ingestion/`.

Loading — `document_loader.py`

DocumentLoader reads PDF (via PyMuPDF), TXT, and DOCX files into plain text plus metadata. It infers the regulation type straight from the filename — a file with “gdpr” in its name is tagged `document_type: GDPR`. That tag later powers filtered search (“only look in HIPAA”).

Chunking — `chunker.py`

Documents are too long to embed whole, so they are split into chunks. Naive fixed-size splitting cuts sentences in half and destroys meaning. Instead, **SemanticChunker** splits on natural boundaries — section breaks, then paragraphs, then sentences — in priority order, with a small overlap between chunks so context isn’t lost at the seams. **RegulationChunker** extends it: it detects article/section headers (e.g. “Article 17”, “Section 164.502”, “CC6.1”) and attaches the legal references it finds to each chunk’s metadata.

Lesson the author learned the hard way

“Chunking strategy matters more than the model.” Retrieval quality was the bottleneck, not generation. Switching from fixed-size to boundary-aware chunking improved how often the right text was found. No amount of agent sophistication fixes bad retrieval.

Embedding & storing — `vector_store.py`

Each chunk is encoded into a 384-dim vector by all-MiniLM-L6-v2 and written to a **ChromaDB** collection configured for cosine similarity. Metadata (filename, `document_type`, section, legal references) is stored alongside each vector so results can be filtered and cited. ChromaDB persists to `./data/chroma_db` on disk, so ingestion only happens once.

Input formats	PDF, TXT, DOCX
Splitting	Boundary-aware (section → paragraph → sentence) with overlap
Embedding model	all-MiniLM-L6-v2 (384 dimensions, runs locally)
Storage	ChromaDB persistent collection, cosine similarity
Metadata kept	filename, <code>document_type</code> , section, <code>legal_references</code> , char offsets

05 Step 2 — Retrieval: Hybrid Search with Citations

When a question arrives, the retriever's job is to surface the handful of chunks most likely to contain the answer. Code lives in `src/retrieval/retriever.py`.

Why hybrid, not pure semantic

Semantic search alone struggles with exact references. Ask for “Article 17 of GDPR” and pure vector similarity tends to surface broadly related material about data rights — not the specific article. Keyword matching catches the literal “Article 17” that semantics miss. **HybridRetriever** runs semantic search first, then nudges the ranking up for chunks that also contain the query's keywords. Semantic relevance leads; keyword matches act as a tie-breaker so exact references float to the top.

The retrieval strategies

- **semantic** — pure nearest-vector search in ChromaDB.
- **hybrid** (default) — semantic results, then a light keyword boost for chunks that contain the query's meaningful terms (stop-words removed).
- **keyword** — a BM25-style scorer that counts term frequencies and blends them with the semantic score (roughly 60% semantic, 40% keyword).

Adding citations — CitationRetriever

CitationRetriever wraps the hybrid retriever and, for every result, builds a human-readable citation from its metadata — e.g. `[1] GDPR - Article 17... (Source: GDPR_Sample.txt)`. It also keeps the raw cosine similarity for ranking while presenting a rescaled, more intuitive 0–1 relevance score in the UI. These citation strings are what later appear under each answer, so every claim links back to a specific source.

Why this matters for the whole project

Citations aren't decoration — they are the product. The entire reason the system exists is that compliance answers must be traceable. Retrieval is where that traceability is manufactured: every chunk carries its origin, and that origin rides all the way to the final answer.

06 Step 3 — The Agent: A LangGraph State Machine

This is the heart of the project. The agent is built as a **LangGraph StateGraph** — an explicit state machine with nodes (steps) and edges (transitions). Code lives in `src/agents/compliance_agent.py`.

The shared state

Every node reads and writes one shared **ComplianceState** object that travels through the graph. It carries the query, the retrieved docs, the draft answer, the reflection text, and the iteration counter — so each step builds on the last.

The nodes

- **retrieve** — calls the hybrid retriever, stores the top-k chunks in state.
- **generate** — builds a system prompt that demands citations and forbids speculation, appends the question and the retrieved context, and calls gpt-4o-mini. The LLM can also request **tools** here.
- **reflect** — a second LLM call that reviews the draft: are claims cited? Any speculation? Contradictions? Anything missing? It returns a short critique.
- **tools** — a ToolNode exposing helper tools (search a specific article, cross-reference regulations, extract requirements, etc.).

The edges — how it decides

The flow isn't fixed; conditional edges route based on state. After **generate**, the agent checks: did the LLM ask to call a tool? → go to **tools**, then back to generate. Otherwise → go to **reflect**. After **reflect**, it checks whether the critique flagged problems (words like “missing”, “incomplete”, “unsupported”). If so and the iteration budget isn't spent, it loops back to **retrieve** to try again; otherwise it ends.

```
+-----+
entry --> | retrieve |
+-----+
      |
      v
+-----+      tool call?      +-----+
| generate | -----> | tools |
+-----+ <-----+ +-----+
      |
      | no tool call
      v
+-----+      needs work? & budget left
| reflect | -----> back to retrieve
+-----+
      |
      | good enough OR max iterations
      v
END -> final answer + citations
```


Why LangGraph instead of a plain LangChain agent

The older agent APIs don't give fine-grained control over the execution loop. LangGraph lets you define explicit state transitions and conditional edges — exactly what the self-reflection loop needs. Without it, the agent either always reflects or never does. With it, reflection fires only when the answer fails a quality check.

Two agents, one system

There's also a **SimpleComplianceAgent**: a one-shot retrieve-then-answer path with no graph and no reflection, selectable via `use_simple_agent: true`. It's the baseline — useful for fast answers and for showing, by contrast, what the reflection loop adds.

07 Step 4 — Memory & the Knowledge Graph

Two optional subsystems give the agent context beyond a single question. Both are designed to degrade gracefully — if the service isn't running, the system keeps working.

Redis — conversation memory

RedisMemory (in `src/memory/redis_memory.py`) stores each session's messages under a key like `session:<id>:messages`, with a one-hour TTL. Fast key-value lookup means conversation history is cheap to read back. Crucially, if Redis isn't reachable it silently falls back to **ConversationBufferMemory**, a plain in-process dictionary — single-session use works with zero infrastructure.

Neo4j — the compliance knowledge graph

Compliance data is naturally a graph: Regulations *contain* Articles, Articles *specify* Requirements, Articles *cross-reference* each other across regulations. **ComplianceKnowledgeGraph** (in `neo4j_graph.py`) models exactly that with node types (Regulation, Article, Requirement, Entity) and typed relationships (CONTAINS, SPECIFIES, REFERENCES).

Why a graph and not a relational table? A question like “what do *both* GDPR and HIPAA say about breach notification?” is a multi-level join in SQL but a simple two-hop traversal in Neo4j. Like Redis, it has a dictionary-based fallback if the database is offline, so graph features simply switch off rather than crash the app.

A design principle the author would keep

“Redis and Neo4j add real operational complexity.” Early versions crashed if either service was down. Adding graceful degradation — in-memory fallback for Redis, disabling graph features when Neo4j is absent — is what made the system usable without the full stack. The takeaway: design optional services as optional from day one.

08 Step 5 — Serving It: API & UI

The agent is wrapped in a REST API and a web UI so it's usable by both machines and people.

FastAPI — `src/api/main.py`

On startup the API builds the vector store, retriever, tools, agent, and memory once and keeps them in memory. It exposes a small, focused set of endpoints:

Method & Path	What it does
POST /query	Ask a compliance question; returns answer, sources, citations, iterations
GET /search	Raw chunk search with optional document_type filter
GET /conversation/{id}	Fetch a session's message history
DELETE /conversation/{id}	Clear a session
GET /stats	Collection stats (chunk count, embedding dim, active sessions)
GET /regulations	List supported regulations
GET /health	Component-by-component health check
POST /ingest	Ingest a new document in the background

The /query handler ties everything together: it records the user message in memory, picks the full agent or the simple agent, runs it, stores the assistant reply, and returns a structured response.

Streamlit — `ui/streamlit_app.py`

The UI is the friendly front door: a text box for the question, a Submit button, and a results panel that shows the answer with its source document and article reference underneath. A sidebar surfaces settings and system stats, and a history view shows past turns. It talks to the FastAPI server over HTTP — the same endpoints any other client would use.

How the pieces run together

1) `python setup.py` loads documents into ChromaDB. 2) `python src/api/main.py` starts the API on :8000. 3) `streamlit run ui/streamlit_app.py` opens the UI on :8501. Redis and Neo4j are optional; without them the system uses its fallbacks.

09 Evaluation: Measuring Quality with RAGAS

A compliance tool you can't measure is a compliance tool you can't trust. The project includes a **RAGAS**-based evaluation pipeline (in `src/evaluation/ragas_eval.py`) that runs a fixed set of test questions with known-good answers and scores the system on four axes.

Metric	The question it answers
Faithfulness	Are answers grounded in the retrieved docs, not hallucinated?
Answer Relevancy	Does the answer actually address the question asked?
Context Recall	Are the right chunks being retrieved in the first place?
Citation Accuracy	Are the article/section references correct?

The test set covers real questions across all three regulations — the right to erasure under GDPR, Protected Health Information under HIPAA, the five Trust Services Criteria in SOC2 — each paired with a ground-truth answer. Running `python ragas_eval.py` produces the scores. Faithfulness doubles as the signal the reflection loop uses: when retrieved chunks score poorly against the draft, the agent retries.

10 Design Decisions & Lessons

The interesting part of any build is the *why*. These are the choices that shaped the system and the things that turned out to be hard.

Decisions

Why LangGraph over plain LangChain agents

Fine-grained control over the execution loop. Explicit state transitions and conditional edges are what make a *conditional* reflection step possible.

Why hybrid retrieval

Pure semantic search missed exact article references. Blending in keyword matching and reranking the combined results noticeably improved precision on targeted queries.

Why Neo4j for the knowledge graph

Cross-regulation relationships are graph traversals, not table joins. Two-hop queries stay simple where SQL would need multi-level joins.

Why Redis for session memory

Fast key-value history per session, with graceful in-memory fallback so the system runs without it.

What was hard

- **Making the reflection loop terminate reliably.** Defining “good enough” was the crux. Too strict and it loops forever; too loose and it never catches a bad answer. RAGAS faithfulness against a threshold became the exit signal — and tuning that threshold took many runs.
- **Tuning hybrid retrieval.** Merging two ranked lists without reranking just gives you two noisy lists stapled together. A weighted combination that favours semantic similarity but boosts exact article-number matches worked best.
- **Operational complexity of Redis and Neo4j.** Retrofitting graceful degradation taught the lesson to design optional services as optional from the start.
- **Chunking beats model tuning.** Time spent on the LLM was wasted while retrieval was the bottleneck; boundary-aware chunking moved the needle far more.

What's next

- Streaming responses, so long agent chains feel responsive instead of blocking.
- A Docker Compose setup to bring up API + Redis + Neo4j + ChromaDB in one command.
- Drag-and-drop document upload in the UI for non-technical users.
- API authentication — fine locally, a blocker for any real deployment.

11 Glossary

Agentic	A system that decides its own next step (e.g. whether to reflect and retry) rather than running a fixed pipeline.
Embedding	A fixed-length numeric vector that captures the meaning of a piece of text. Here, 384 dimensions.
Vector store	A database (ChromaDB) that stores embeddings and answers nearest-neighbour queries by similarity.
Cosine similarity	A measure of how close two vectors point in the same direction; used to rank relevance.
RAG	Retrieval-Augmented Generation — retrieve relevant text, then have the LLM answer using it.
Hybrid retrieval	Combining semantic (vector) search with keyword matching, then reranking the merged results.
BM25	A classic keyword-relevance scoring formula based on term frequency.
Chunk	A bounded piece of a document, split on natural boundaries, that gets embedded and stored.
LangGraph	A framework for building agents as explicit state machines with nodes and conditional edges.
StateGraph / node / edge	The graph (StateGraph), its steps (nodes), and the transitions between them (edges).
Reflection	A second LLM pass that critiques the draft answer and can trigger a retry.
Citation	A source reference (regulation, article/section, filename) attached to each retrieved chunk and answer.
Knowledge graph	A graph of entities and typed relationships (Neo4j) modelling how regulations relate.
Graceful degradation	Continuing to work with reduced features when an optional service (Redis/Neo4j) is unavailable.
RAGAS	An evaluation framework scoring RAG systems on faithfulness, relevancy, recall, and precision.
gpt-4o-mini	The OpenAI model used for generation and reflection — fast, cheap, strong reasoning.

You now have the full arc: the problem, the concepts, the pipeline stage by stage, and the reasoning behind every major choice. The single thread tying it together — retrieve, ground, cite, and self-check — is what turns a chatbot into a compliance auditor you can actually trust.
